Module 8: Arrays

8.1 Introduction to Arrays

- An array is a collection of a fixed number of elements all of the same data type stored in contiguous memory locations
- Instead of declaring 100 separate integer variables, declare one array of 100 integers
- All elements are accessed using the same name but different index numbers
- Elements are stored in consecutive memory addresses

8.2 Declaring and Initializing Arrays

- Declaration syntax: data_type arrayName[size];
  int scores[5]; declares an array that can hold 5 integers
  The size must be a constant known at compile time (for standard arrays)
- Initializing at declaration with values in braces
  int scores[5] = {90, 85, 78, 92, 88};

- Partial initialization: providing fewer values than the size
  - Remaining elements are automatically initialized to zero
  - int nums[5] = {1, 2}; creates {1, 2, 0, 0, 0}
- Declaring without specifying size when initializing
  - int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
  - Compiler counts the values and sets the size to 12

8.3 Accessing Array Elements

- Each element is accessed using its index (position number)
- Array indexing starts at 0
- First element: arrayName[0]
- Second element: arrayName[1]
- Last element of size n: arrayName[n-1]
- Reading an element: cout << scores[0]; prints 90
- Modifying an element: scores[2] = 95;
- Out-of-bounds access
- Accessing an index outside the valid range (below 0 or equal to or beyond the size) is undefined behavior
- The compiler does not check this at runtime; it may read garbage or crash the program
- Always ensure your index is between 0 and size minus 1

8.4 Traversing Arrays with Loops

- A for loop is the standard way to process all elements of an array

```
for (int i = 0; i < 5; i++) {
cout << scores[i] << " ";
}
```

- Reading values from the user into an array

```
for (int i = 0; i < 5; i++) {
cout << "Enter score " << i+1 << ": ";
cin >> scores[i];
}
```

- Calculating the sum of all elements

```
int total = 0;
for (int i = 0; i < 5; i++) {
total += scores[i];
}
double average = (double)total / 5;
```

- Finding the minimum and maximum value

```
int minVal = scores[0], maxVal = scores[0];
for (int i = 1; i < 5; i++) {
if (scores[i] < minVal) minVal = scores[i];
if (scores[i] > maxVal) maxVal = scores[i];
}
```

8.5 Passing Arrays to Functions

- Arrays are always passed to functions by reference (the address of the first element is passed)
- Changes made to the array inside the function affect the original array
- The size of the array is not passed automatically; it must be passed as a separate parameter

```
void printArray(int arr[], int size) {
for (int i = 0; i < size; i++) {
cout << arr[i] << " ";
}
}
printArray(scores, 5);
```

8.6 Multidimensional Arrays

- A 2D array is like a table with rows and columns
- Declaration: int matrix[3][4]; (3 rows, 4 columns, holds 12 elements total)
- Initialization

```
int grid[2][3] = { {1, 2, 3}, {4, 5, 6} };
```

- Accessing a specific element: grid[0][2] is 3 (row 0, column 2)
- Traversing with nested loops

```
for (int row = 0; row < 2; row++) {
for (int col = 0; col < 3; col++) {
cout << grid[row][col] << " ";
}
cout << endl;
}
```

- Applications: storing a matrix, representing a chessboard, storing a table of data

8.7 Common Array Operations

- Linear search: scan each element one by one to find a target value
- Bubble sort: repeatedly compare adjacent elements and swap if out of order, repeat until sorted
- Reversing an array: swap elements from both ends moving toward the center
- Copying an array: loop through and copy each element individually (assignment operator does not copy arrays)